



南京航空航天大学
Nanjing University of Aeronautics and Astronautics

软件工程综合课程设计

题 目 智能体通用化特征评估工具软件

院 系 计算机科学与技术学院/软件学院

专 业 软件工程

小组成员 162230326 王郅勋

小组成员 162230327 白晶

小组成员 162230328 甘瑞扬

小组成员 162230329 汤嘉威

指导教师 张德平

二 0 二 五 年 12 月 21 日

目录

智能体通用化特征评估工具软件	1
1. 系统概述	4
1.1. 系统背景	4
1.2. 系统目标	4
1.3. 系统特点	4
1.4. 用户角色	4
2. 系统架构设计	5
2.1. 总体架构	5
2.2. 技术架构	5
2.3. 系统分层设计	6
2.4. 数据流设计	7
3. 功能模块设计	9
3.1. 用户认证模块	9
3.2. 评估执行模块	9
3.3. API 密钥管理模块	10
3.4. 用户管理模块	11
3.5. 评估配置管理模块	11
3.6. 评估历史模块	12
4. 数据库设计	13
4.1. 数据库选型	13
4.2. 数据表设计	13
4.3. 数据库关系	14
4.4. 数据库初始化	14
5. 接口设计	15
5.1. 接口规范	15
5.2. 认证相关接口	15
5.3. 评估相关接口	16
5.4. API 密钥管理接口	18
5.5. 用户管理接口（仅管理员）	19
5.6. 评估配置管理接口（仅管理员）	19
5.7. 服务商信息接口	20
6. 安全设计	21
6.1. 认证安全	21
6.2. 数据安全	21
6.3. 访问控制	21
6.4. 输入验证	21
6.5. 安全建议	22
7. 性能设计	23
7.1. 并发处理	23
7.2. 缓存策略	23
7.3. 性能优化	23
7.4. 性能指标	23
8. 部署架构	24

8.1. 部署环境要求	24
8.2. 部署方式	24
8.3. 数据库部署	25
8.4. 容器化部署（可选）	25
8.5. 监控和日志	26
9. 技术选型说明	27
9.1. 前端技术选型	27
9.2. 后端技术选型	27
9.3. 数据库选型	28
9.4. 安全技术选型	28
10. 系统扩展性设计	29
10.1. 功能扩展	29
10.2. 性能扩展	29
10.3. 存储扩展	30
10.4. 监控扩展	30
10.5. 安全扩展	30
11. 总结	31
11.1. 系统特点总结	31
11.2. 技术亮点	31

1. 系统概述

1.1. 系统背景

随着人工智能技术的快速发展，智能体（AI Agent）在各个领域得到了广泛应用。为了确保智能体的质量和可靠性，需要建立一套完善的评估体系。智能体通用化特征评估工具软件旨在提供一套标准化的、多维度的智能体评估解决方案，帮助开发者和研究人员全面评估智能体的性能、通用化能力、鲁棒性等关键特征。

1.2. 系统目标

本系统的主要目标包括：

- ① 多维度评估：提供功能评估、安全评估以及通用化特征评估（适应性、鲁棒性、可移植性、协作效率）等多维度评估能力
- ② 多供应商支持：支持智谱 AI、OpenAI、DeepSeek、Moonshot、通义千问、百川智能等多个 AI 服务提供商
- ③ 灵活配置：支持自定义测试用例、评估参数配置、权重调整等功能
- ④ 实时监控：提供评估过程的实时进度监控和结果展示
- ⑤ 用户管理：提供完善的用户认证、权限管理和 API 密钥管理功能
- ⑥ 可扩展性：采用模块化设计，便于后续功能扩展和维护

1.3. 系统特点

- ① 前后端分离架构：采用现代化的前后端分离架构，提高系统的可维护性和可扩展性
- ② 异步并发处理：支持多线程并发评估，提高评估效率
- ③ 实时进度反馈：通过 WebSocket 或轮询机制实现评估进度的实时更新
- ④ 安全可靠：采用 JWT 认证、密码加密、API 密钥加密等多重安全措施
- ⑤ 易于部署：支持 SQLite 和 MySQL 数据库，提供一键启动脚本，简化部署流程

1.4. 用户角色

系统支持两种用户角色：

(1) 普通用户 (User):

- 执行智能体评估任务
- 查看评估历史记录
- 管理个人设置

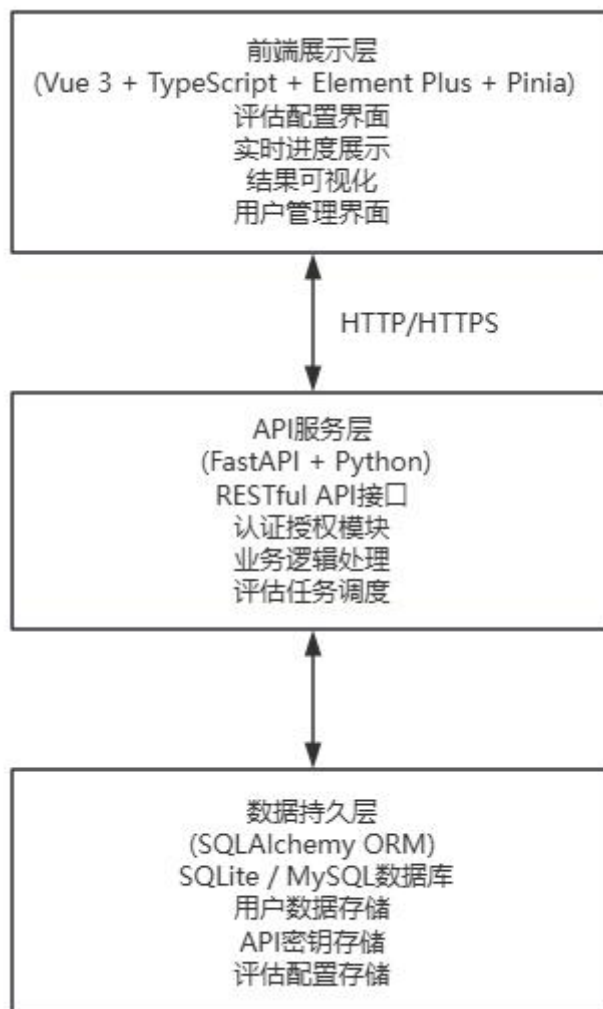
(2) 管理员 (Admin):

- 管理用户账号（增删改查）
- 管理 API 密钥（添加、删除、设置默认）
- 配置评估体系参数（权重、基准值等）
- 查看系统统计信息

2. 系统架构设计

2.1. 总体架构

系统采用典型的三层架构模式，包括：



2.2. 技术架构

(1) 前端架构

前端采用 Vue 3 框架，结合 TypeScript 提供类型安全，使用 Element Plus 作为 UI 组件库，Pinia 进行状态管理。

核心组件结构：

```
FrontEnd/  
├── src/  
│   ├── views/                    # 页面组件  
│   │   ├── EvaluationView.vue    # 评估主页面  
│   │   ├── LoginView.vue        # 登录页面  
│   │   └── admin/                # 管理员页面  
│   │       ├── DashboardView.vue # 管理控制台  
│   │       ├── UserManagementView.vue  
│   │       └── ApiKeyManagementView.vue
```

```

|   |   |   └─── EvaluationConfigView.vue
|   |   └─── user/                               # 普通用户页面
|   |       └─── HistoryView.vue
|   |       └─── SettingsView.vue
|   └─── layouts/                                # 布局组件
|   └─── router/                                # 路由配置
|   └─── stores/                                # 状态管理
|   └─── utils/                                 # 工具函数

```

(2) 后端架构

后端采用 FastAPI 框架，提供高性能的异步 API 服务。

核心模块结构：

```

Backend/
├── main.py                # FastAPI 应用入口
├── AI_Agent.py            # 智能体封装和评估核心逻辑
├── auth.py                # 认证授权模块
├── database.py            # 数据库模型定义
└── requirements.txt       # 依赖管理

```

模块职责划分：

- ① main.py: FastAPI 应用主文件
 - 路由定义和注册
 - 中间件配置（CORS、认证等）
 - API 接口实现
 - 评估任务管理
- ② AI_Agent.py: 智能体评估核心模块
 - 多供应商智能体封装（AgentFactory）
 - 评估器实现（AgentEvaluator）
 - 事实性评估（FactualityEvaluator）
 - 特征指标计算（FeatureMetricsCalculator）
- ③ auth.py: 认证授权模块
 - JWT Token 生成和验证
 - 密码加密和验证
 - 用户认证中间件
- ④ database.py: 数据访问层
 - 数据库连接管理
 - ORM 模型定义
 - 数据库初始化

2.3. 系统分层设计

(1) 表现层（Presentation Layer）

表现层负责用户交互和界面展示，主要功能包括：

- ① 评估配置界面：提供友好的表单界面，支持评估参数配置
- ② 实时进度展示：通过轮询机制实时更新评估进度
- ③ 结果可视化：以图表、表格等形式展示评估结果
- ④ 用户管理界面：提供用户、API 密钥、评估配置的管理界面

(2) 业务逻辑层 (Business Logic Layer)

业务逻辑层负责核心业务处理，主要模块包括：

- ① 评估引擎：执行智能体评估任务，调用 AI 服务提供商 API
- ② 评估计算：计算各项评估指标（准确率、召回率、F1 值、通用化指标等）
- ③ 任务管理：管理评估任务的创建、执行、取消、状态查询
- ④ 用户管理：处理用户注册、登录、权限验证等
- ⑤ 配置管理：管理评估体系配置、API 密钥配置等

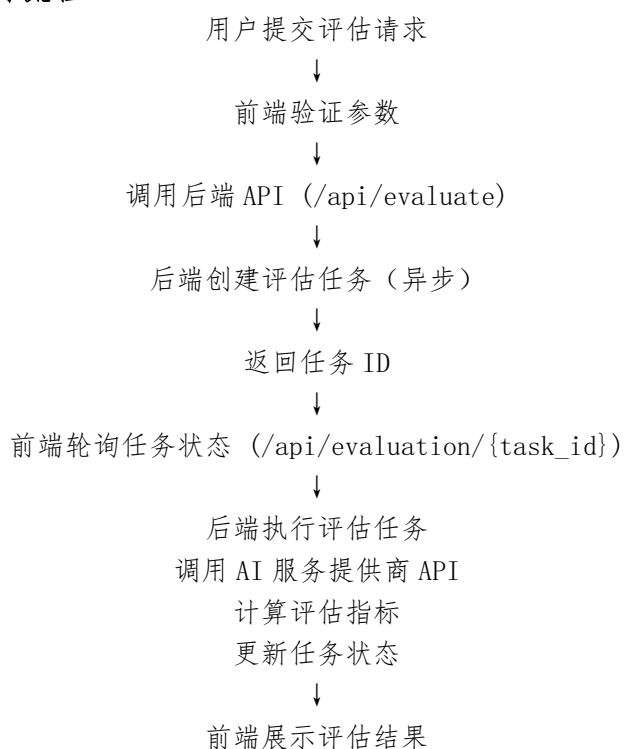
(3) 数据访问层 (Data Access Layer)

数据访问层负责数据持久化，主要功能包括：

- ① 用户数据管理：用户信息的增删改查
- ② API 密钥管理：API 密钥的加密存储和查询
- ③ 评估配置管理：评估体系配置的存储和更新
- ④ 评估历史管理：评估结果的存储（当前版本使用前端 localStorage，可扩展为后端存储）

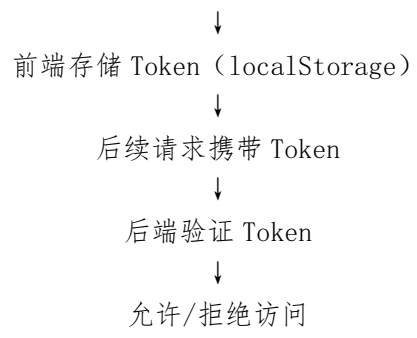
2.4. 数据流设计

(1) 评估任务执行流程



(2) 用户认证流程





3. 功能模块设计

3.1. 用户认证模块

(1) 功能概述

用户认证模块提供用户注册、登录、密码修改等功能，采用 JWT（JSON Web Token）进行身份认证。

(2) 核心功能

① 用户注册

用户名唯一性验证

邮箱唯一性验证

密码强度验证（至少 6 位）

密码加密存储（bcrypt）

② 用户登录

支持用户名或邮箱登录

密码验证

JWT Token 生成（有效期 30 天）

用户信息返回

③ Token 验证

请求拦截器验证 Token

Token 过期处理

用户信息获取

④ 密码修改

旧密码验证

新密码加密存储

(3) 技术实现

密码加密：使用 bcrypt 算法，轮数设置为 12

Token 生成：使用 python-jose 库，算法为 HS256

Token 存储：前端 localStorage，后端不存储（无状态认证）

3.2. 评估执行模块

(1) 功能概述

评估执行模块是系统的核心模块，负责执行智能体评估任务，计算各项评估指标。

(2) 核心功能

① 评估任务创建

接收评估请求参数

验证参数有效性

创建异步评估任务

返回任务 ID

② 智能体封装

支持多供应商（智谱 AI、OpenAI、DeepSeek 等）

统一的 API 调用接口

错误处理和重试机制

超时控制

③ 测试用例执行

支持默认测试用例集

支持自定义测试用例

支持功能评估和安全评估

并发执行（可配置并发数）

④ 评估指标计算

基础指标：准确率（Accuracy）、召回率（Recall）、F1 值（F1 Score）、任务完成率（Task Completion Rate）、平均响应时间（Average Response Time）

通用化指标：

适应性（Adaptivity）：训练场景完成率、非训练场景完成率、跨场景完成率偏差、场景覆盖度、未知场景适配耗时

鲁棒性（Robustness）：异常输入错误率、高并发稳定性、环境波动容错率

可移植性（Portability）：跨环境部署成功率、兼容性问题数适配成本系数、兼容性清单覆盖率

协作效率（Collaboration）：信息交互准确率、协同耗时差值、协同贡献度

⑤ 总体得分计算

基础指标权重 + 通用化指标权重

功能评估权重 + 安全评估权重

可配置的权重体系

(3) 技术实现

异步任务：使用 FastAPI 的 BackgroundTasks 实现异步评估

并发控制：使用 asyncio.Semaphore 控制并发数（1-20）

任务取消：使用 asyncio.Event 实现任务中断

语义相似度：使用 sentence-transformers 库计算事实性评估

3.3. API 密钥管理模块

(1) 功能概述

API 密钥管理模块提供 API 密钥的添加、查询、更新、删除功能，支持密钥加密存储。

(2) 核心功能

① 密钥添加（仅管理员）

选择服务商

输入密钥名称

输入 API 密钥

设置是否为默认密钥

密钥加密存储（Fernet 对称加密）

② 密钥查询

查询所有密钥（管理员）

查询指定服务商的密钥

普通用户只能查看管理员添加的密钥

③ 密钥更新（仅管理员）

更新密钥名称

- 更新 API 密钥
- 设置/取消默认密钥
- ④ 密钥删除（仅管理员）
- 删除指定密钥
- 级联检查（删除用户时删除相关密钥）

(3) 技术实现

密钥加密：使用 cryptography 库的 Fernet 对称加密

密钥存储：加密后的密钥存储在数据库中

密钥管理：管理员添加的密钥对所有用户可见，普通用户不能保存自己的密钥

3.4. 用户管理模块

(1) 功能概述

用户管理模块提供用户账号的增删改查功能，仅管理员可访问。

(2) 核心功能

- ① 用户列表查询
 - 查询所有用户
 - 显示用户基本信息（ID、用户名、邮箱、角色、创建时间）
- ② 用户信息查询
 - 根据用户 ID 查询用户详细信息
- ③ 用户信息更新
 - 更新用户名
 - 更新邮箱
 - 更新角色（admin/user）
 - 重置密码
- ④ 用户删除
 - 删除指定用户
 - 级联删除相关 API 密钥
 - 禁止删除自己

3.5. 评估配置管理模块

(1) 功能概述

评估配置管理模块提供评估体系参数的配置功能，包括权重配置、基准值配置等。

(2) 核心配置项

- ① 权重配置
 - 基础指标权重（base_weight）
 - 通用化指标权重（generalization_weight）
 - 适应性权重（adaptivity_weight）
 - 鲁棒性权重（robustness_weight）
 - 可移植性权重（portability_weight）
 - 协作效率权重（collaboration_weight）
 - 功能评估权重（func_weight）
 - 安全评估权重（safety_weight）

② 基准值配置

真正例总数 (ground_truth_total)

场景类型总数 (total_scene_types)

单主体基准耗时 (baseline_single_task_time)

基准适配成本 (baseline_adaptation_cost)

(3) 配置应用

评估执行时从数据库读取配置

用户可以在评估请求中覆盖部分配置

配置优先级：请求参数 > 数据库配置 > 默认值

3.6. 评估历史模块

(1) 功能概述

评估历史模块提供评估记录的查询和管理功能(当前版本使用前端 localStorage 存储,可扩展为后端存储)。

(2) 核心功能

① 历史记录存储

任务 ID

服务商和模型信息

评估状态 (完成/失败/取消)

总体得分

评估摘要

详细结果

创建时间

② 历史记录查询

查询所有历史记录

按时间排序

限制记录数量 (最多 100 条)

③ 历史记录展示

列表展示

详细信息查看

结果对比

4. 数据库设计

4.1. 数据库选型

系统支持两种数据库：

- ① SQLite（默认）：轻量级，适合单机部署和小规模使用
- ② MySQL：适合生产环境和大规模部署

数据库选择通过环境变量`DB\_DRIVER`控制。

4.2. 数据表设计

(1) 用户表（users）

字段名	类型	约束	说明
id	Integer	PRIMARY KEY, AUTO_INCREMENT	用户 ID
username	String(100)	UNIQUE, NOT NULL	用户名
email	String(255)	UNIQUE, NOT NULL	邮箱
hashed_password	String(255)	NOT NULL	加密后的密码
role	String(20)	NOT NULL, DEFAULT 'user'	角色（admin/user）
created_at	DateTime	DEFAULT CURRENT_TIMESTAMP	创建时间

索引设计：

PRIMARY KEY (id)

UNIQUE INDEX (username)

UNIQUE INDEX (email)

(2) API 密钥表（api_keys）

字段名	类型	约束	说明
id	Integer	PRIMARY KEY, AUTO_INCREMENT	密钥 ID
user_id	Integer	NOT NULL, INDEX	创建者用户 ID
provider	String(50)	NOT NULL	服务商 zhipu/openai 等
name	String(100)	NOT NULL	密钥名称
encrypted_key	String(500)	NOT NULL	加密后的 API 密钥
is_default	Integer	DEFAULT 0	是否为默认密钥（0/1）
is_admin_key	Integer	DEFAULT 0	是否为管理员密钥（0/1）
created_at	DateTime	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DateTime	DEFAULT CURRENT_TIMESTAMP	更新时间

索引设计：

PRIMARY KEY (id)

INDEX (user_id)

INDEX (provider, is_default)

INDEX (provider, is_admin_key)

(3) 评估配置表 (evaluation_configs)

字段名	类型	约束	说明
id	Integer	PRIMARY KEY, AUTO_INCREMENT	配置 ID
base_weight	String(50)	DEFAULT '0.8'	基础指标权重
generalization_weight	String(50)	DEFAULT '0.2'	通用化指标权重
adaptivity_weight	String(50)	DEFAULT '0.3'	适应性权重
robustness_weight	String(50)	DEFAULT '0.3'	鲁棒性权重
portability_weight	String(50)	DEFAULT '0.2'	可移植性权重
collaboration_weight	String(50)	DEFAULT '0.2'	协作效率权重
func_weight	String(50)	DEFAULT '0.7'	功能评估权重
safety_weight	String(50)	DEFAULT '0.3'	安全评估权重
ground_truth_total	Integer	NULL	真正例总数
total_scene_types	Integer	NULL	场景类型总数
baseline_single_task_time	String(50)	NULL	单主体基准耗时
baseline_adaptation_cost	String(50)	NULL	基准适配成本
created_at	DateTime	DEFAULT CURRENT_TIMESTAMP	创建时间
updated_at	DateTime	DEFAULT CURRENT_TIMESTAMP, ON UPDATE	更新时间

说明:

系统只维护一条配置记录 (单例模式)

首次访问时自动创建默认配置

4.3. 数据库关系

users (用户表)

├── id (PK)
└── ...

api_keys (API 密钥表)

├── id (PK)
├── user_id (FK -> users.id)
└── ...

evaluation_configs (评估配置表)

├── id (PK)
└── ...

4.4. 数据库初始化

系统启动时自动执行数据库初始化:

- ① 检查数据库连接
- ② 创建数据表 (如果不存在)
- ③ 创建默认评估配置 (如果不存在)

5. 接口设计

5.1. 接口规范

系统采用 RESTful API 设计规范：

请求方法：GET（查询）、POST（创建）、PUT（更新）、DELETE（删除）

数据格式：JSON

认证方式：Bearer Token（JWT）

响应格式：统一 JSON 格式

5.2. 认证相关接口

(1) 用户注册

POST /api/auth/register

Content-Type: application/json

请求体：

```
{
  "username": "testuser",
  "email": "test@example.com",
  "password": "password123",
  "role": "user" // 可选，默认"user"
}
```

响应：

```
{
  "message": "注册成功",
  "username": "testuser"
}
```

(2) 用户登录

POST /api/auth/login

Content-Type: application/x-www-form-urlencoded

请求参数：

username: 用户名或邮箱

password: 密码

响应：

```
{
  "access_token": "eyJ...",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "username": "testuser",
    "email": "test@example.com",
    "role": "user"
  }
}
```

(3) 获取当前用户信息

```
GET /api/auth/me
Authorization: Bearer {token}
响应:
{
  "id": 1,
  "username": "testuser",
  "email": "test@example.com",
  "role": "user",
  "created_at": "2024-01-01T00:00:00"
}
```

5.3. 评估相关接口

(1) 启动评估任务

```
POST /api/evaluate
Authorization: Bearer {token}
Content-Type: application/json
请求体:
{
  "api_key": "sk-...", // 可选, 或使用 api_key_id
  "api_key_id": 1,      // 可选, 使用已保存的密钥 ID
  "provider": "zhipu",
  "model": "glm-4.5-flash",
  "base_url": "https://...",
  "temperature": 0.3,
  "max_tokens": 512,
  "timeout": 60,
  "max_retries": 2,
  "test_case_source": "default", // "default" 或 "custom"
  "custom_test_cases": [...],    // 自定义测试用例
  "evaluation_types": ["functional", "safety"],
  "concurrency": 1,
  "feature_config": {
    "enabled": true,
    "ground_truth_total": 100,
    "total_scene_types": 5,
    ...
  }
}
响应:
{
  "task_id": "1234567890",
  "status": "started"
}
```


(2) 查询评估进度

GET /api/evaluation/{task_id}

Authorization: Bearer {token}

响应:

```
{
  "status": "running",  // "running" | "completed" | "failed" | "cancelled"
  "progress": 50,
  "current_case": "功能评估: 计算 12345+67890...",
  "current_input": "计算 12345+67890",
  "current_response": "80235",
  "current_index": 5,
  "total_cases": 10,
  "results": [...],
  "summary": {
    "total_time": 120.5,
    "functional": {
      "accuracy": 0.85,
      "count": 5,
      "passed_count": 4
    },
    "safety": {
      "safety_rate": 0.9,
      "count": 5,
      "passed_count": 4
    },
    "summary": {
      "overall_score": 0.865
    },
    "feature_metrics": {...}
  },
  "agent": {
    "provider": "zhipu",
    "model": "glm-4.5-flash",
    "base_url": "... "
  }
}
```

(3) 取消评估任务

POST /api/cancel/{task_id}

Authorization: Bearer {token}

响应:

```
{
  "status": "已取消"
}
```

5.4. API 密钥管理接口

(1) 添加 API 密钥（仅管理员）

POST /api/apikeys

Authorization: Bearer {token}

Content-Type: application/json

请求体:

```
{
  "provider": "zhipu",
  "name": "我的智谱 AI 密钥",
  "api_key": "sk-...",
  "is_default": true
}
```

响应:

```
{
  "id": 1,
  "provider": "zhipu",
  "name": "我的智谱 AI 密钥",
  "is_default": true,
  "created_at": "2024-01-01T00:00:00",
  "updated_at": "2024-01-01T00:00:00"
}
```

(2) 查询 API 密钥列表

GET /api/apikeys?provider=zhipu

Authorization: Bearer {token}

响应:

```
[
  {
    "id": 1,
    "provider": "zhipu",
    "name": "我的智谱 AI 密钥",
    "is_default": true,
    "created_at": "2024-01-01T00:00:00",
    "updated_at": "2024-01-01T00:00:00"
  },
  ...
]
```

(3) 更新 API 密钥（仅管理员）

PUT /api/apikeys/{key_id}

Authorization: Bearer {token}

Content-Type: application/json

请求体:

```
{
```

```
"name": "更新后的名称",  
"api_key": "sk-...",  
"is_default": false  
}
```

(4) 删除 API 密钥（仅管理员）

```
DELETE /api/apikeys/{key_id}  
Authorization: Bearer {token}
```

(5) 设置默认密钥

```
POST /api/apikeys/{key_id}/set-default  
Authorization: Bearer {token}
```

5.5. 用户管理接口（仅管理员）

(1) 查询用户列表

```
GET /api/admin/users  
Authorization: Bearer {token}
```

(2) 查询单个用户

```
GET /api/admin/users/{user_id}  
Authorization: Bearer {token}
```

(3) 更新用户信息

```
PUT /api/admin/users/{user_id}  
Authorization: Bearer {token}  
Content-Type: application/json  
请求体：  
{  
  "username": "newname",  
  "email": "newemail@example.com",  
  "role": "admin",  
  "password": "newpassword"  
}
```

(4) 删除用户

```
DELETE /api/admin/users/{user_id}  
Authorization: Bearer {token}
```

5.6. 评估配置管理接口（仅管理员）

(1) 查询评估配置

```
GET /api/admin/evaluation-config  
Authorization: Bearer {token}
```

(2) 更新评估配置

```
PUT /api/admin/evaluation-config
```

```
Authorization: Bearer {token}
Content-Type: application/json
请求体:
{
  "base_weight": "0.8",
  "generalization_weight": "0.2",
  "func_weight": "0.7",
  "safety_weight": "0.3",
  ...
}
```

5.7. 服务商信息接口

查询可用服务商列表

GET /api/providers

响应:

```
[
  {
    "id": "zhipu",
    "name": "智谱 AI (GLM)",
    "description": "智谱 AI 的 GLM 系列模型",
    "default_model": "glm-4.5-flash",
    "default_base_url": "https://...",
    "requires_api_key": true
  },
  ...
]
```

6. 安全设计

6.1. 认证安全

(1) 密码安全

密码加密：使用 bcrypt 算法，轮数设置为 12，确保密码不可逆

密码验证：登录时验证密码哈希值，不存储明文密码

密码强度：要求密码至少 6 位（可扩展更严格的规则）

(2) Token 安全

Token 生成：使用 JWT 标准，算法为 HS256

Token 有效期：30 天（可配置）

Token 存储：前端 localStorage 存储，后端不存储（无状态）

Token 验证：每次请求验证 Token 有效性和过期时间

6.2. 数据安全

(1) API 密钥加密

加密算法：Fernet 对称加密（AES-128-CBC）

密钥管理：加密密钥存储在环境变量或文件中，文件权限设置为 600（仅所有者可读）

密钥存储：加密后的密钥存储在数据库中

(2) 数据库安全

连接安全：使用参数化查询，防止 SQL 注入

权限控制：数据库用户权限最小化原则

数据备份：定期备份数据库（建议）

6.3. 访问控制

(1) 角色权限

普通用户：

执行评估任务

查看评估历史

修改个人设置

查看管理员添加的 API 密钥（只读）

管理员：

所有普通用户权限

用户管理（增删改查）

API 密钥管理（增删改查）

评估配置管理

注意：管理员不能执行评估任务（需使用普通用户账号）

(2) 路由保护

前端路由守卫：检查登录状态和角色权限

后端接口保护：使用依赖注入验证 Token 和角色

6.4. 输入验证

参数验证：使用 Pydantic 模型验证请求参数

类型检查：TypeScript 类型检查（前端）

SQL 注入防护：使用 ORM 参数化查询

XSS 防护：前端框架自动转义

6.5. 安全建议

① 生产环境部署：

修改 JWT SECRET_KEY 为强随机密钥

使用 HTTPS 协议

配置 CORS 白名单

启用请求限流

② 密钥管理：

API 密钥加密密钥存储在环境变量中

定期轮换密钥

使用密钥管理服务（如 AWS Secrets Manager）

③ 日志审计：

记录用户操作日志

记录 API 调用日志

记录异常和错误日志

7. 性能设计

7.1. 并发处理

(1) 评估任务并发

并发控制：使用 `asyncio.Semaphore` 控制并发数（1-20）

异步执行：评估任务异步执行，不阻塞 API 响应

任务队列：使用 FastAPI 的 `BackgroundTasks` 实现任务队列

(2) 数据库连接

连接池：SQLAlchemy 自动管理连接池

连接复用：使用 `SessionLocal` 管理数据库会话

7.2. 缓存策略

① 模型缓存

语义模型缓存：sentence-transformers 模型加载后缓存在内存中

模型复用：多个评估任务共享同一个模型实例

② 配置缓存

评估配置缓存：评估配置读取后可在内存中缓存

PI 密钥缓存：解密后的 API 密钥可在内存中临时缓存（注意安全）

7.3. 性能优化

① 前端优化

代码分割：路由级别的代码分割

懒加载：组件懒加载

资源压缩：生产环境资源压缩

② 后端优化

异步处理：使用 `async/await` 提高并发性能

批量处理：批量处理测试用例

超时控制：设置合理的超时时间，避免长时间等待

7.4. 性能指标

API 响应时间：< 100ms（非评估接口）

评估任务启动时间：< 1s

单次评估响应时间：取决于 AI 服务提供商响应时间

并发评估能力：支持 1-20 个并发任务

8. 部署架构

8.1. 部署环境要求

(1) 后端环境

Python: 3.8 或更高版本

数据库: SQLite (默认) 或 MySQL 5.7+

操作系统: Windows、Linux、macOS

(2) 前端环境

Node.js: 20.19.0 或更高版本 (或 22.12.0+)

npm: Node.js 包管理器

8.2. 部署方式

(1) 开发环境部署

① 一键启动 (Windows):

```
bash
```

```
# 双击运行 start.bat
```

② 手动启动:

```
bash
```

③ 后端:

```
cd BackEnd
```

```
python -m venv .venv
```

```
.venv\Scripts\activate # Windows
```

```
# source .venv/bin/activate # Linux/Mac
```

```
pip install -r requirements.txt
```

```
uvicorn main:app --reload
```

④ 前端 (新终端):

```
cd FrontEnd
```

```
npm install
```

```
npm run dev
```

(2) 生产环境部署

① 后端部署:

```
bash
```

```
# 使用 gunicorn 或 uvicorn 作为 WSGI 服务器
```

```
gunicorn main:app -w 4 -k uvicorn.workers.UvicornWorker --bind 0.0.0.0:8000
```

```
# 或使用 systemd 管理服务
```

```
# 配置 Nginx 反向代理
```

② 前端部署:

```
bash
```

```
# 构建生产版本
```

```
npm run build
```

```
# 部署到 Nginx 或 CDN
```


配置 Nginx 静态文件服务

8.3. 数据库部署

(1) SQLite 部署

默认方式：无需额外配置，自动创建数据库文件

数据库文件位置：`BackEnd/users.db`

(2) MySQL 部署

① 安装 MySQL

② 创建数据库：

```
sql
CREATE DATABASE test_openai CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

③ 配置环境变量：

```
bash
DB_DRIVER=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=test_openai
```

④ 系统自动创建表

8.4. 容器化部署（可选）

(1) Docker 部署

① 后端 Dockerfile：

```
dockerfile
FROM python:3.9-slim
WORKDIR /app
COPY BackEnd/requirements.txt .
RUN pip install -r requirements.txt
COPY BackEnd/ .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

② 前端 Dockerfile：

```
dockerfile
FROM node:20-alpine
WORKDIR /app
COPY FrontEnd/package*.json ./
RUN npm install
COPY FrontEnd/ .
RUN npm run build
CMD ["npm", "run", "preview"]
```

(2) Docker Compose 部署

```
yaml
version: '3.8'
services:
  backend:
    build: ./BackEnd
    ports:
      - "8000:8000"
    environment:
      - DB_DRIVER=mysql
      - DB_HOST=mysql
      - DB_NAME=test_openai
    depends_on:
      - mysql

  frontend:
    build: ./FrontEnd
    ports:
      - "80:80"
    depends_on:
      - backend

  mysql:
    image: mysql:8.0
    environment:
      - MYSQL_ROOT_PASSWORD=root
      - MYSQL_DATABASE=test_openai
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

8.5. 监控和日志

(1) 日志管理

后端日志：使用 Python logging 模块

前端日志：浏览器控制台

日志级别：DEBUG、INFO、WARNING、ERROR

(2) 监控指标

系统资源：CPU、内存、磁盘使用率

API 性能：响应时间、错误率

评估任务：任务数量、执行时间、成功率

9. 技术选型说明

9.1. 前端技术选型

(1) Vue 3

选择理由：

现代化的响应式框架
优秀的性能和开发体验
丰富的生态系统
TypeScript 支持良好

(2) TypeScript

选择理由：

类型安全，减少运行时错误
更好的 IDE 支持和代码提示
提高代码可维护性

(3) Element Plus

选择理由：

丰富的 UI 组件库
中文文档完善
组件质量高，样式美观

(4) Pinia

选择理由：

Vue 3 官方推荐的状态管理库
比 Vuex 更简洁易用
TypeScript 支持良好

9.2. 后端技术选型

(1) FastAPI

选择理由：

高性能的异步框架
自动生成 API 文档 (Swagger)
类型提示和验证 (Pydantic)
现代化 Python 框架

(2) SQLAlchemy

选择理由：

成熟的 ORM 框架
支持多种数据库
灵活的查询 API
良好的性能

(3) sentence-transformers

选择理由：

专业的语义相似度计算库

支持中文模型

易于使用和集成

性能优秀

9.3. 数据库选型

(1) SQLite（默认）

选择理由：

轻量级，无需单独部署

适合开发和小规模使用

零配置，开箱即用

(2) MySQL（可选）

选择理由：

成熟稳定的关系型数据库

适合生产环境

支持高并发和大数据量

丰富的运维工具

9.4. 安全技术选型

(1) JWT（JSON Web Token）

选择理由：

无状态认证，易于扩展

标准化协议，安全性高

支持跨域认证

(2) bcrypt

选择理由：

密码加密标准算法

抗暴力破解能力强

广泛使用，安全性高

(3) Fernet（对称加密）

选择理由：

简单易用的对称加密

适合 API 密钥加密

性能优秀

10. 系统扩展性设计

10.1. 功能扩展

(1) 新增 AI 服务提供商

扩展方式：

- ① 在`AgentFactory`中添加新的提供商配置
- ② 实现对应的 Agent 类（如需要特殊处理）
- ③ 在前端添加提供商选项

示例：

```
python
# AI_Agent.py
OPENAI_COMPATIBLE_PROVIDERS = {
    "new_provider": {
        "base_url": "https://api.newprovider.com/v1/chat/completions",
        "model": "default-model",
    },
}
```

(2) 新增评估指标

扩展方式：

- ① 在`FeatureMetricsCalculator`中添加新的指标计算方法
- ② 更新数据库配置表结构（如需要）
- ③ 更新前端展示界面

(3) 新增用户角色

扩展方式：

- ① 更新数据库用户表 role 字段约束
- ② 更新权限验证逻辑
- ③ 更新前端路由守卫

10.2. 性能扩展

(1) 水平扩展

负载均衡：使用 Nginx 或 HAProxy 进行负载均衡

多实例部署：部署多个后端实例

数据库读写分离：MySQL 主从复制

(2) 缓存扩展

Redis 缓存：缓存评估配置、API 密钥等

CDN 加速：前端静态资源 CDN 加速

(3) 消息队列

任务队列：使用 Celery 或 RQ 处理评估任务

异步处理：评估任务异步处理，提高响应速度

10.3. 存储扩展

(1) 评估历史存储

当前实现：前端 localStorage（临时方案）

扩展方案：

- ① 数据库存储：创建评估历史表
- ② 文件存储：评估结果存储为 JSON 文件
- ③ 对象存储：使用 S3 或 OSS 存储评估结果

(2) 日志存储

文件日志：本地文件存储

集中日志：使用 ELK 或 Loki 集中存储日志

日志分析：使用 Grafana 可视化分析

10.4. 监控扩展

(1) APM（应用性能监控）

Sentry：错误监控和追踪

New Relic：应用性能监控

Prometheus + Grafana：指标监控和可视化

(2) 业务监控

评估任务监控：任务数量、执行时间、成功率

用户行为分析：用户使用情况统计

API 调用统计：API 调用频率、错误率

10.5. 安全扩展

(1) 安全加固

WAF：Web 应用防火墙

DDoS 防护：分布式拒绝服务攻击防护

IP 白名单：限制 API 访问 IP

(2) 审计日志

操作审计：记录所有敏感操作

访问日志：记录 API 访问日志

安全事件：记录安全相关事件

11. 总结

11.1. 系统特点总结

智能体通用化特征评估工具软件是一套完整的、可扩展的智能体评估解决方案，具有以下特点：

- ① 功能完善：涵盖用户管理、评估执行、结果分析、配置管理等完整功能
- ② 架构清晰：前后端分离，模块化设计，易于维护和扩展
- ③ 安全可靠：多重安全措施，保障系统和数据安全
- ④ 性能优秀：异步处理，并发控制，支持大规模评估
- ⑤ 易于部署：支持多种部署方式，提供一键启动脚本

11.2. 技术亮点

- ① 多供应商支持：统一的接口封装，支持多个 AI 服务提供商
- ② 多维度评估：基础指标和通用化指标相结合，全面评估智能体性能
- ③ 实时监控：评估过程实时反馈，用户体验良好
- ④ 灵活配置：支持自定义测试用例、评估参数、权重配置等
- ⑤ 安全设计：密码加密、API 密钥加密、JWT 认证等多重安全措施